

PES UNIVERSITY

Department of Computer Science and Engineering

SIMPLE SHELL – COMMAND LINE INTERPRETER

Software Engineering Level 3 – Synopsis / Project Proposal

Group Number	Team 3
Project ID	TBD / To be assigned
Technology	C
Methodology	Agile

1. Team Profile

Sl.	USN	SRN / PRN	Name	Functional Ownership
1	PES1UG24CS370	PES1202403285	Rekha Dhorigol	Command Interpretation & Core Built-in Commands
2	PES1UG24CS382	PES1202402229	Rohan Rampura Praveen Kumar	Process Execution & Job Control
3	PES1UG24CS412	PES1202402146	Samika Ojha	I/O Redirection & Pipelines
4	PES1UG24CS421	PES1202402156	Sanjana Subrahmanya Aithal	Command History & Shell Environment

2. Problem Statement

Command-line shells provide an efficient interface for interacting with an operating system, but their internal operation involves several important concepts such as command parsing, process creation, program execution, file descriptors, inter-process communication and environment management. For students and users learning operating-system concepts, the behaviour of a shell can appear as a single opaque operation. The project addresses this by developing a modular Simple Shell in C that implements essential command-line functionality while explicitly demonstrating the underlying operating-system mechanisms.

3. Proposed Project Description

The proposed system is a lightweight command-line interpreter implemented in C for a Linux/Unix-like environment. It will repeatedly display a prompt, accept user commands, parse them, identify built-in and external commands, and execute them. The shell will support built-in commands, external process execution, foreground and background execution, input/output redirection, command pipelines, command history and environment-variable management. The implementation will be modular so that each major functionality can be developed, tested and maintained independently. The project will also demonstrate software-engineering practices including Agile planning, Git/GitHub-based version control, code reviews, automated testing, CI/CD, static analysis and requirement traceability.

4. Target Users / Stakeholders

User / Stakeholder	Need / Role
Students / Learners	Use the shell to learn and experiment with operating-system and command-line concepts.
Command-line Users	Execute common Linux/Unix commands through a lightweight custom shell.
Developers / Testers	Use the modular implementation as a basis for testing process, parsing and IPC behaviour.
Faculty / Evaluators	Evaluate the implementation and the team's application of the software development lifecycle.

5. Objectives

- Develop a functional command-line interpreter in C for a Linux/Unix-like environment.

- Implement robust command parsing and a defined set of built-in commands.
- Create and manage child processes for execution of external commands.
- Support foreground/background execution and appropriate process and signal handling.
- Implement input/output redirection using file descriptors.
- Implement single and multiple command pipelines using inter-process communication.
- Provide command history and shell environment-variable management.
- Handle invalid commands, malformed syntax and runtime errors without crashing the shell.
- Develop the system incrementally using Agile practices with measurable, testable requirements.
- Validate the system through unit, integration, system, security/robustness and non-functional testing.

6. Scope

6.1 In Scope

- Interactive command prompt and command-line input handling.
- Tokenization and parsing of commands and arguments.
- Built-in commands such as cd, pwd, echo, exit, export, unset, env and help.
- Execution of external programs using the POSIX process model.
- Foreground and background command execution.
- Input redirection (<), output redirection (>) and append redirection (>>).
- Single and multiple command pipelines using the pipe (|) operator.
- Command history storage and display, with a defined command-recall mechanism.
- Environment-variable access and session-level management.
- Error handling, input validation, memory-safety checks and robustness testing.

6.2 Out of Scope

- Graphical user interface.
- Remote shell access or network-based command execution.
- Full-featured shell scripting language compatibility.
- Shell-specific features not explicitly listed in the approved requirements.
- Administrative/root-level system management.
- A persistent multi-user database.

7. Functional Features

ID	Feature	Description
F1	Command Interpretation & Core Built-in Commands	Reads user input, tokenizes and parses commands and arguments, classifies built-in and external commands, dispatches commands to the appropriate subsystem, executes supported core built-in commands, and reports invalid input.
F2	Process Execution & Job Control	Executes external programs using process creation and program replacement, supports foreground/background execution and handles relevant process signals.
F3	I/O Redirection & Pipelines	Redirects standard input/output to files and connects commands through one or more pipelines using file descriptors and IPC.
F4	Command History & Shell Environment	Stores and displays previous commands and provides session-level access to environment variables through supported shell operations.

8. Team-wise Functional Ownership

Each member owns one complete functional area. Supporting activities such as documentation, testing, Agile planning, code review and CI/CD are shared across the team and do not replace functional ownership.

Member	Owned Functionality	Key Technical Areas
Rekha Dhorigol	Command Interpretation & Core Built-in Commands	C strings/pointers, input handling, tokenization, command parsing, command classification, command dispatch, input validation
Rohan Rampura Praveen Kumar	Process Execution & Job Control	fork, exec, waitpid, PIDs, signals, foreground/background execution
Samika Ojha	I/O Redirection & Pipelines	file descriptors, open/close, dup2, pipe, IPC
Sanjana Subrahmanya Aithal	Command History & Shell Environment	history data structures, getenv/setenv/unsetenv, session state

9. Plan of Work

Sprint	Planned Work / Output
Sprint 1	Project setup, requirements baseline, use cases, architecture planning and basic shell loop.
Sprint 2	Command parser and built-in command implementation.
Sprint 3	External process execution, foreground/background execution and signal handling.
Sprint 4	Input/output redirection and single/multiple pipelines.
Sprint 5	Command history, environment management, integration, security/robustness testing and CI/CD refinement.
Sprint 6	System validation, defect fixing, coverage/static-analysis evidence, final documentation and demonstration.

10. Proposed Technology / Tools

Area	Technology / Tool
Language	C
Operating Environment	Linux (POSIX-compatible environment)
Compiler / Build	GCC and Make
Version Control	Git and GitHub
Agile Management	Jira or GitHub Projects
Testing	C unit-testing framework such as GoogleTest/Check, plus shell-based integration tests as appropriate
CI/CD	Jenkins
Containerization	Docker
Static Analysis	SonarQube or a suitable C static-analysis tool

11. Agile Justification

Agile is proposed because the shell can be developed incrementally as independently demonstrable functional modules. Each sprint will produce a working increment, followed by unit/integration testing, code review and refinement. The backlog will contain user stories and technical tasks, while GitHub and Jira/GitHub Projects will provide evidence of ownership, sprint progress and issue tracking. Requirements will be baselined early but refined only through controlled changes so that later design, implementation and testing remain traceable.

12. Expected Outcomes

- A working C-based interactive Simple Shell executable.
- Modular implementation covering parsing, built-ins, process execution, redirection/pipelines, history and environment management.
- Documented requirements, use cases, architecture/design and functional ownership.
- Unit, integration and system test evidence with coverage and validation results.
- GitHub/Jira activity, pull requests, code reviews and CI/CD evidence.
- A final requirement-to-test traceability chain and demonstration-ready release.

13. Conclusion

The Simple Shell project aims to combine practical operating-system programming with a disciplined software-engineering development process. By dividing the system into complete functional modules and developing them incrementally, the team will be able to demonstrate clear individual ownership while maintaining an integrated working product. The final system will provide essential shell capabilities and serve as a practical demonstration of processes, system calls, file descriptors, inter-process communication and environment management.